7/8/26 **Exp-6 Error detection & Correction**

Aim:-

To write a Program to implement error detection and Correction using Hamming Code.

Algorithm:-

① Start the program

② Read the number of 4 bit data blocks (n) forming the input data stream.

③ For each 4 bit data block $D = d_1 d_2 d_3 d_4$, Perform Hamming encoding:

ⓐ Arrange 7 bits with Parity bits at Positions 1, 2 & 4 and data bits at Position 3, 5, 6 and 7

ⓑ Compute the Parity bits using user Parity:

$p_1 = d_1$ Xor d_2 Xor d_4 (correct bit-1,3,5,7)

$p_2 = d_1$ Xor d_3 Xor d_4 (correct bit-2,3,6,7)

$p_4 = d_2$ Xor d_3 Xor d_4 (correct bit-4,5,6,7)

ⓒ Form the 7 bit hamming code word:

$p_1 \, p_2 \, d_1 \, p_4 \, d_2 \, d_3 \, d_4$.

④ Simulate transmission by introducing a single bit error at a chosen bit position in the code word.

⑤ At receiver, recompute the check bits from received code word:

$c_1 = p_1$ Xor d_1 Xor d_2 Xor d_4

$c_2 = p_2$ XOR d_1 XOR d_3 XOR d_4

$c_4 = p_4$ XOR d_2 XOR d_3 XOR d_4

⑥ Compute Error Position = $c_4 \, c_2 \, c_1$ (binary no formed)

=== HAMMING CODE ERROR DETEC
AND CORRECTION

```
Enter no of 4 bit data block to tran
--- Block 1 ---
Enter 4 data bits (space separated, eg
1011): 0101
original Data bits : 0101
Encoded Hamming code (7 bits): 01
Enter bit position to introduce
error (1-7, 0 = No error): 3
Transmitted code (with error induced
at pos 3): 0110101
Received code: 0110101
Recovered data bits: 0101
```

⑦ If error Position = 0, no error has occurred. Else, bit at "Error Position" is inverted to correct the error

⑧ Extract the original 4 data bits from corrected 7 bits code.

⑨ Repeat steps 3 to 8 for all n blocks in the data stream

⑩ Stop the Program.

Program:-

```
def encode (d):
  """
    d: list of 4 bits [d1, d2, d3, d4]
    hamming code list position (1-indexed)
  1 = p1, 2 = p2, 3 = d1, 4 = p4, 5 = d2, 6 = d3,
          """
    7 = d4
    h = [0] * 7
    h[2] = d[0]
    h[4] = d[1]
    h[5] = d[2]
    h[6] = d[3]
    h[0] = h[2] ^ h[4] ^ h[6]
    h[1] = h[2] ^ h[5] ^ h[6]
    h [3] = h[4] ^ h[5] ^ h[6]
    return h
def detect and correct (h):
  c1 = h[0] ^ h[2] ^ h[4] ^ h[6]
  c2 = h[1] ^ h[2] ^ h[5] ^ h[6]
  c4 = h [3] ^ h[4] ^ h[5] ^ h[6]
error_pos = c1*1 + c2*2 + c4*4
```

Scanned with OKEN Scanner

```
if error_pos != 0:
    Print ("Error detected at bit position:", error_pos)
    h[error_pos-1] ^= 1
else:
    Print ("No error detected")
return error_pos
def to_str(bits):
    return "".join(str(b) for b in bits)

def main():
    n = int(input())
    for b in range(1, n+1):
        Print (f"Block {b}")
        d = list(map(int, input().split()))
        h = encode(d)
        Print(f"original data bits: {to_str(d)}")
        Print (f"Encoded Hamming code
(7 bits): {to_str(h)}")
        pos = int(input("Enter bit to
introduce error (1-7), 0 = No error"))
        if 1 <= pos <= 7:
            h[pos-1] ^= 1
            Print(f"Transmitted code: {to_str(h)}")
        else:
            Print(f"Transmitted code: {to_str(h)}")
    Print(f"Received code: {to_str(h)}")
    Print("Checking error")
    detect_error_and_correct(h)
    Print(f"Corrected code: {to_str(h)}")
    rd = [h[2], h[4], h[5], h[6]]
    Print(f"recovered data bits: {to_str(rd)}")
if __name__ == "main":
    main()
```

Result:-

The Program to Implement error detection & Correction using hamming Code Concept was Successfully written in Python, executed and Verified with a Sample Input data Stream.

Scanned with OKEN Scanner